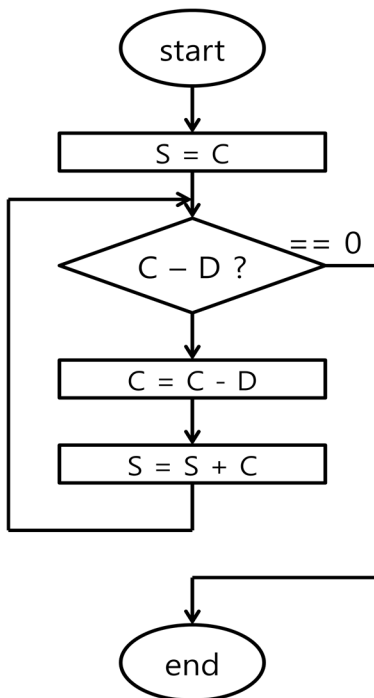


# 컴퓨터구조(2026 봄) 학기말 숙제

숙제#07에서 제시된 명령어 세트의 모든 명령어를 수행하는 CPU를 완성하고(\*.circ파일 업로드), 실행되는 프로그램을 작성하라.(소스코드 \*.asm파일, RAM 내용 \*.img 파일 업로드)

| opcode | mnemonic |                        | opcode | mnemonic |                           |
|--------|----------|------------------------|--------|----------|---------------------------|
| 0x0    | NOP      | No operation           | 0x8    | LDA addr | Load M[addr] to AC        |
| 0x1    | NOT      | Negate AC              | 0x9    | ADD addr | Add M[addr] to AC         |
| 0x2    | SHL      | Shift left AC          | 0xA    | SUB addr | Subtract M[addr] from AC  |
| 0x3    | SHR      | Shift right AC         | 0xB    | AND addr | Bitwise AND M[addr] to AC |
| 0x4    | JMP addr | Jump to addr           | 0xC    | OR addr  | Bitwise OR M[addr] to AC  |
| 0x5    | BZ addr  | Branch on zero to addr | 0xD    | XOR addr | Bitwise XOR M[addr] to AC |
| 0x6    | (unused) |                        | 0xE    | (unused) |                           |
| 0x7    | (unused) |                        | 0xF    | STR addr | Store AC to M[addr]       |



1부터 10까지 더하는 프로그램을 만들어서, 직접 만든 CPU에서 실행시켜보라.

좌측의 흐름도는 1부터 10까지 더하는 프로그램의 흐름도이다. 초기조건으로 각 변수를 S = 0, C = 10, D = 1 으로 설정하고, 흐름도를 실행하면 1부터 10까지 더한 값이 S에 저장된다.

이를 참조하여 위 표의 니모닉을 사용한 어셈블리 언어로 프로그램을 작성한 후, opcode 값을 보고 16진수로 손으로 변환하여(hand-assemble) RAM에 저장한 후 실행시켜보라. (코드는 0번지부터, 변수는 FF번지부터 사용하기 추천)

어셈블리 소스코드를 일반 텍스트 파일 포맷으로 \*.asm 확장자 붙인 뒤 업로드, RAM의 내용을 \*.img 파일로 저장하여 업로드 한다.

**Bonus 1:** 사용하지 않는 opcode 값 0x6을 이용하여 Z 플래그가 0일 때 주어진 주소로 분기하는 BNZ(branch on non-zero) 명령을 추가하라.

**Bonus 2:** 모든 명령의 실행에 8클럭이 걸리는 것은 비효율적이다. 불필요한 uNOP 마이크로 연산이 실행되지 않도록, 각 명령의 실행 사이클 단축하라. (명령의 종류에 따라 실행에 5클럭, 혹은 7클럭이 걸리도록 수정하라.)

**Bonus 3:** LDA, ADD, SUB, AND, OR, XOR 명령에 즉시 주소지정방식(immediate addressing mode)를 추가하라.

opcode가 LDA, ADD, SUB, AND, OR, XOR(0x8 ~ 0xD)이고, 명령 첫 번째 바이트의 3번 비트(그림의 'M'비트)가 1인 경우를 immediate mode로 간주한다.



**Bonus 4:** 기존의 명령 세트와 bonus1, 3에서 추가된 BNZ명령, 즉시 주소지정방식의 명령 이용하여, 변수 'D'가 필요 없고, BZ 명령을 사용하지 않는, 1부터 10까지 더하는 프로그램을 작성하라. (흐름도부터 그려보는 것 추천)

**Bonus 5:** 1부터 100까지 더한 값은 5050이고, 이는 1바이트로는 표현할 수 없다. 2바이트 이상의 정수를 1바이트씩 계산하려면, 낮은 자리수의 1바이트를 더했을 때 발생하는 carry 값을, 높은 자리수의 1바이트의 덧셈에 이용하는 명령이 필요하다. 이를 ADC(add with carry) 명령이라 하자. 덧셈 수행 시, 현재의 상태 레지스터의 carry bit 값을 포함하여 덧셈하는 ADC명령을, 미사용중인 op code 0xE에 할당하여 구현하고, 이를 이용하여 루프를 돌며 1부터 100까지 더하는 프로그램을 작성하라. (hint: LDA명령 수행시에는 status register를 변경시키지 않게해야 함)